

Symbolic and Neural Maze Solving

André Silva, Ricardo Santana

May 2025

Abstract

Solving mazes is a classical optimization problem addressed by search algorithms. Despite impressive gains in knowledge benchmarks, Large Language Models (LLMs) have been challenged by spatial reasoning tasks. On the other hand, symbolic solvers have long been known to excel in long-term planning tasks. To benchmark the limitations of LLMs against symbolic solvers, we focus on the task of random maze solving, using 2D mazes (from 7x7 to 32x32 size grids), and compare the performance of SOTA reasoning and non-reasoning LLMs to classical solvers. Our findings show that neither type of LLMs can perfectly solve mazes like their symbolic counterparts. Reasoning LLMs can find solutions for some large maze sizes (up to 29x29), but are extremely time-consuming compared to classical methods. Moreover, prompting the models with annotated historical trials improves performance, but models still face problems handling large mazes.

1 Introduction

The challenge of navigating mazes represents a classical optimization problem in Artificial Intelligence, typically addressed by symbolic search algorithms. These methods are well-established for their ability to systematically explore the solution space and, in many cases, guarantee optimal paths. However, they lack an inherent understanding of semantic nuances or contextual information [4, 5].

Unlike symbolic algorithms, LLMs lack innate spatial representations. They rely on textual descriptions of the environment to infer geometric relationships and constraints. Early works have found that, while LLMs can plan short paths under specific prompting conditions, they are challenged by long-term planning, consistency, and the need to avoid 'hallucinations' in complex scenarios [2, 1, 3].

Conversely, symbolic solvers have a long record of excelling in long-term planning tasks where the problem structure is well-defined. This contrast between the data-driven, reasoning capabilities of LLMs and the structured, verifiable planning of symbolic methods presents an interesting research avenue. The potential combination of these paradigms could lead to the development of more contextually aware and interpretable planning engines, with significant applications in fields such as robotics, where agents must navigate and interact with complex environments and humans (e.g., [6]).

To rigorously benchmark the capabilities and limitations of recent LLMs against established symbolic solvers, we focus on the task of random 2D maze solving in this work. We investigate the performance of state-of-the-art reasoning and non-reasoning LLMs on grid-based mazes of varying dimensions (ranging from 7x7 to 32x32) and compare their efficacy and computational demands against classical search algorithms. Our work specifically explores LLMs' processing capabilities on spatial information, their scalability to larger problem instances, and the impact of different input representations and prompting strategies, including the use of annotated historic trials, on their planning performance. The findings from our comparative analysis reveal that while reasoning-augmented LLMs are capable of successfully finding solutions in surprisingly large mazes (up to 29x29 in our experiments), they are substantially more time-consuming and unreliable than classical methods. Furthermore, we demonstrate that prompting strategies incorporating feedback from past attempts can enhance LLM performance in this domain.

2 Methods

2.1 Classical Solvers

We compare two well-established symbolic maze-solving algorithms: the CollisionSolver (a variant of the flood-fill approach) and the Recursive Backtracker. Both algorithms operate on a square grid of dimension $d \times d$, where each cell is either a wall or a traversable corridor.

2.1.1 CollisionSolver

- **Overview:** The CollisionSolver performs a simultaneous, bidirectional flood-fill from the start and end points. At each iteration, it floods all currently reachable free cells one step further in every compass direction.
- **Collision Handling:** When two flood fronts meet at adjacent cells, a wall is placed at their meeting point, preventing further mixing of the two regions.
- **Dead-End Filling:** After each flooding pass, any newly created dead-end corridors (cells with exactly one open neighbor) are filled in as walls.
- **Dead-End Removal:** This removes redundant branches and guides the search toward the shortest path between start and end.
- **Iteration:** Flooding and dead-end filling repeat until no new collisions occur. The remaining open corridors trace the shortest path(s) through the maze.
- **Properties:**
 - Guarantees shortest-path discovery in perfect mazes.
 - It may require multiple global passes in highly braided or imperfect mazes, but still converges to an optimal solution.

2.1.2 Recursive Backtracker

- **Overview:** Also known as the depth-first search maze generator’s solving analogue, this algorithm picks a random direction at each step and moves if the adjacent cell is unvisited and free.
- **Backtracking:** Upon reaching a dead end (no unvisited neighbors), the algorithm backtracks to the most recent cell that has unexplored directions and resumes random exploration.
- **Properties:**
 - Always finds *a* solution, but not necessarily the shortest.
 - Efficient in time ($\mathcal{O}(d^2)$) and space (stack-based recursion), and handles imperfect mazes robustly.
 - It differs from the Random Mouse algorithm solely in that it avoids immediately reversing direction unless at a dead end.

2.2 LLM-Based Solvers

We implement a prompt-driven maze solver using large language models (LLMs). Each maze is encoded in-prompt as a 2D grid of characters, with deterministic rules enforcing valid movements. We experiment with both multi-token ASCII and single-token emoji input representations.

LLM Prompt (ASCII and emoji input representations)

You are given a square maze, represented by a 2D grid of characters:

- Walls: {wall_char} (impassable),
- Free cells: {path_char} (traversable),

Coordinates are 0-indexed: (0,0) is the top-left, ({dim-1},{dim-1}) is the bottom-right.

Start position: {start_point}

End position: {end_point}

Rules:

1. You may move one cell at a time: up, down, left, or right.
2. No diagonal moves.
3. No jumping over {wall_char} cells.
4. You cannot move into {wall_char} cells.
5. You cannot revisit any cell (including the start and end points if part of a longer $\hookrightarrow$ path attempt).
6. You must return a valid path from the start to the end.
7. You must return a complete path, not a partial path.
8. You must NOT return an empty path, there is a solution to the maze.

Output:

- A single Python list of (row, col) tuples, in order from the start to the end, $\hookrightarrow$ including both endpoints.
- The entire list must appear on one line, with no extra text or commentary.

Here is the maze map (each line is one row): {maze_map_str}

Historic Attempt Feedback

To increase test-time computation in non-reasoning models and approximate the way humans solve mazes via trial-and-error, we experiment with few-shot learning by including verification of past LLM trials.

Here is a previous attempt (Attempt #{i}) including all steps until the first invalid $\hookrightarrow$ step ({attempt_path[first_invalid_idx]}) at index {first_invalid_idx}:
 $\hookrightarrow$ path_segment_to_show)

The last point shown was where the path became invalid. Do not repeat this mistake. You

- $\hookrightarrow$ may need to backtrack from earlier valid steps. Do not assume all other steps
- $\hookrightarrow$ before the error are correct parts of the final solution.

2.2.1 Evaluation Regime

We test both symbolic and LLM solvers on a set of randomly generated mazes, varying maze sizes from 3×3 to 15×15 . Each solver’s *solution length*, *computation time*, and *success rate* (valid path returned) are recorded. Performance is averaged over 10 independent maze instances per dimension.

3 Results

Figure 1 shows the success rate of symbolic and LLM-based solvers by maze size. The symbolic solvers are able to solve all mazes regardless of size, while LLM-based solvers struggle as maze size increases.

The symbolic solvers, CollisionSolver and BacktrackingSolver, consistently solve all 130 mazes, achieving a 100% success rate. Their average solve times are extremely low (0.0007s and 0.0028s, respectively), and the path lengths are optimal and identical for both methods.

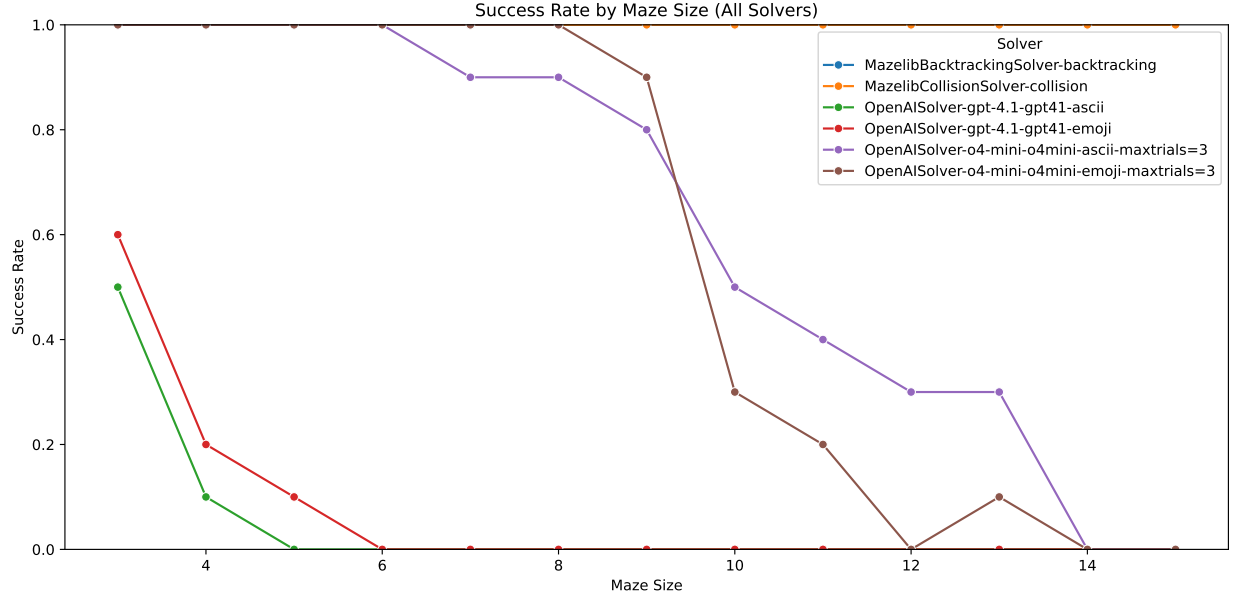


Figure 1: Success rate of symbolic and LLM-based solvers by maze size. The symbolic solvers are able to solve all mazes, while the LLM-based solvers struggle to solve mazes, particularly as the maze size increases. The non-reasoning LLM (GPT-4.1) is unable to solve any maze larger than 11x11. The reasoning LLM (O4-mini), when allowed up to three tries, performs substantially better than in single-trial settings, but still fails on over a third of the mazes and remains much slower than symbolic solvers.

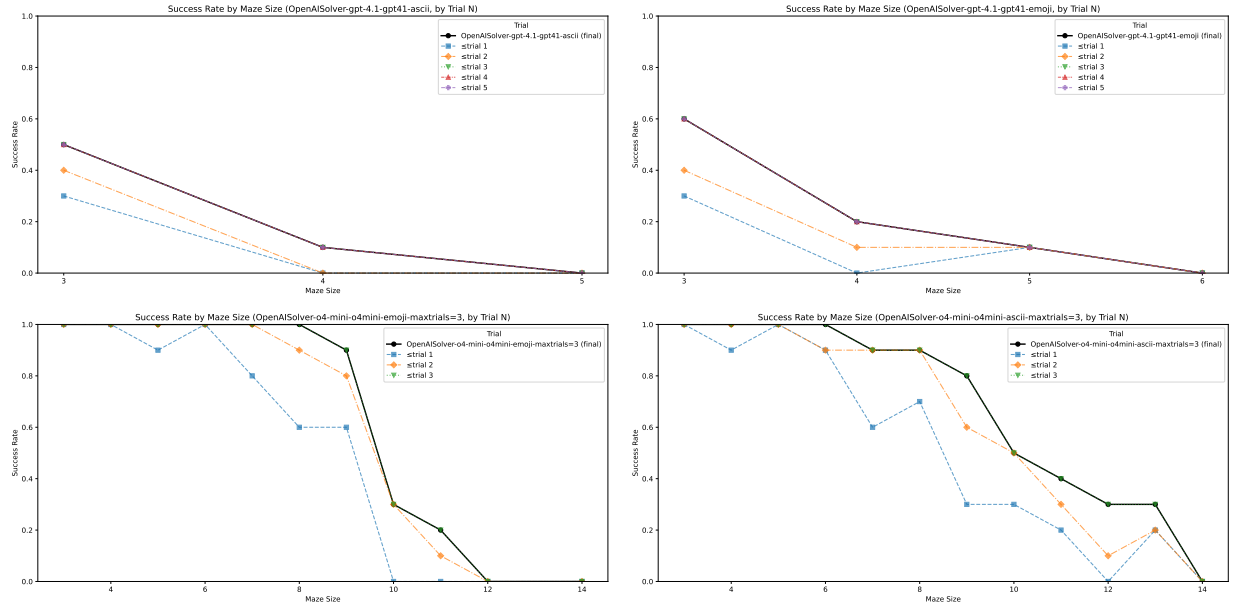


Figure 2: Success rate of the LLM solvers by maze size and across multiple trials. Increasing test-time computation by providing previous attempts to the model and retrying improves performance. However, more than three trials do not yield additional performance gains for the non-reasoning LLM (GPT-4.1), and the model still struggles with larger mazes.

In contrast, the LLM-based solvers show significantly lower performance. The non-reasoning LLM (GPT-

4.1) achieves a success rate of only 4.6% (ASCII) and 6.9% (emoji), with average solve times around 21–22 seconds. The reasoning LLM (O4-mini) achieves a success rate of 62.3% (ASCII) and 57.7% (emoji). The average solve times are also much higher than those for symbolic solvers (240s for ASCII, 154s for emoji).

We also observe that, for LLM solvers, changing the maze representation from ASCII to emoji does not fundamentally change their behavior. For GPT-4.1, the emoji representation is slightly better than ASCII, but for O4-mini, the performance is mixed, with the ASCII representation being better for larger mazes.

Figure 2 shows the success rate of the LLM solvers by maze size and across multiple trials, for both ASCII and emoji maze formats. Allowing models to retry by providing them with their previous failed attempts leads to noticeable improvements in success rate, especially for smaller mazes. We hypothesize that increased test-time computation provides them with the ability to explore multiple alternative paths.

For example, in Figure 3, the reasoning LLM (O4-mini) is able to solve a 13x13 maze only after two trials. The model reaches the end of the maze in the first attempt but traverses impassable walls, requiring a second attempt to find the correct path. The gains from multiple trials, however, plateau after three trials for the non-reasoning LLM (GPT-4.1), and the model remains unable to solve larger mazes effectively, highlighting the limitations of this approach for more complex instances. As for the reasoning LLM (O4-mini), we do not reach saturation with the allowed three trials, but the gains are clearly limited, and the model still fails to solve larger mazes.

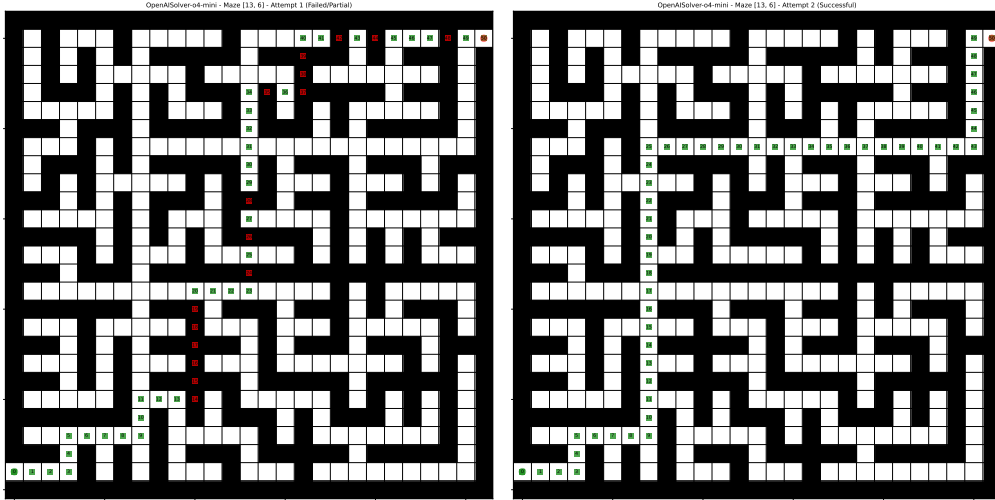


Figure 3: Example of a 28x28 maze solved by the reasoning LLM (o4-mini) in two trials. The model fails to solve the maze in the first attempt, traversing the maze through walls. In the second attempt, the model traverses the maze correctly, showing how the model can learn from its mistakes.

4 Discussion

4.1 Visual Reasoning

Our experiments show that text-based LLMs struggle in solving mazes, even when provided with multiple trials. However, humans typically solve mazes by reasoning about the visual layout, not by text.

We perform a preliminary experiment to test whether vision-language models (VLMs) can solve mazes. Figure 4 shows a failed attempt by ChatGPT (vision mode) to solve a 28x28 maze. The model fails to solve the maze, returning both an invalid path and a different version of the provided maze. This example illustrates the current limitations of VLMs in solving mazes, suggesting that even visual reasoning is not the only missing piece in LLMs’ ability to solve mazes.

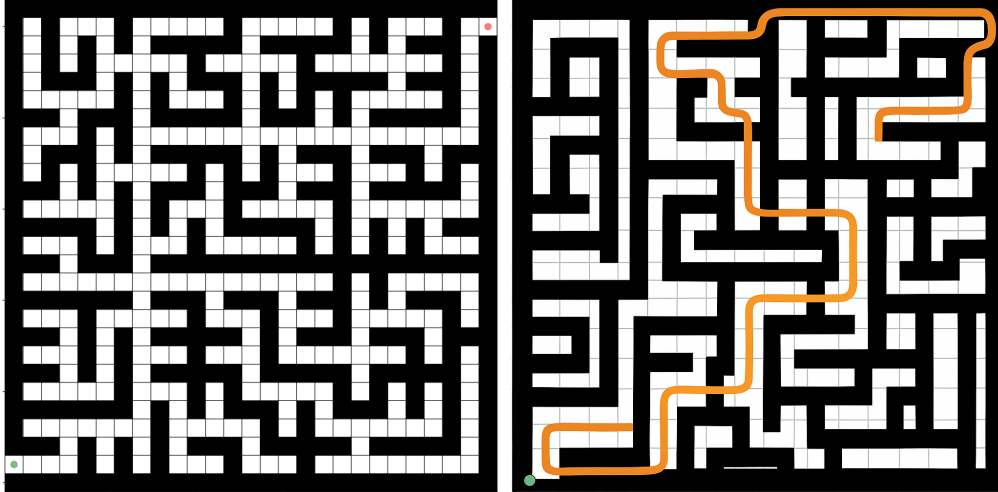


Figure 4: Attempt by ChatGPT (vision mode) to solve a 28x28 maze. On the left, the empty maze to be solved. On the right, the proposed solution. The model fails to solve the maze, returning both an invalid path and a different version of the provided maze.

4.2 Symbolic Solvers vs. LLMs

The stark performance contrast between symbolic and LLM-based solvers in this study underscores a critical consideration in the application of AI: the nature of the problem itself. Mazes, as classical optimization problems, are defined by explicit rules, discrete states, and a verifiable goal. Symbolic solvers like the CollisionSolver and BacktrackingSolver are engineered for such environments, leveraging algorithmic precision to explore the solution space efficiently and guarantee optimality.

5 Conclusion

Our findings highlight the current limitations of LLMs in spatial reasoning tasks, particularly in the context of maze navigation. While reasoning LLMs are able to solve some mazes, they are much slower and less reliable than symbolic solvers, and require significant test-time computation to achieve reasonable performance. These results suggest that, for problems characterized by clear logic and a need for verifiable, optimal solutions, classical symbolic algorithms remain the gold standard.

References

- [1] Mohamed Aghzal, Erion Plaku, and Ziyu Yao. Can large language models be good path planners. *A Benchmark and Investigation on Spatial-temporal Reasoning*, February, 2024.
- [2] Weizhe Chen, Sven Koenig, and Bistra Dilkina. Why solving multi-agent path finding with large language model has not succeeded yet. *arXiv preprint arXiv:2401.03630*, 2024.
- [3] Francois Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. Arc prize 2024: Technical report. *arXiv preprint arXiv:2412.04604*, 2024.
- [4] Panagiotis Giadikiaroglou, Maria Lymperaiou, Giorgos Filandrianos, and Giorgos Stamou. Puzzle solving using reasoning of large language models: A survey. *arXiv preprint arXiv:2402.11291*, 2024.
- [5] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. *Advances in Neural Information Processing Systems*, 36:38975–38987, 2023.

- [6] Hengjia Xiao and Peng Wang. Llm a*: Human in the loop large language models enabled a* search for robotics. *arXiv preprint arXiv:2312.01797*, 2023.